



You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

API Reference

Every plugin component receives a `pluginApi` object that provides access to plugin functionality and Noctalia services.

Plugin API Object

The `pluginApi` object is injected into all plugin components (`BarWidget`, `DesktopWidget`, `ControlCenterWidget`, `LauncherProvider`, `Panel`, `Main`, `Settings`):

```
Item {  
  // Injected by PluginService  
  property var pluginApi: null  
  
  Component.onCompleted: {  
    if (pluginApi) {  
      Logger.i("Plugin", "API available:", pluginApi.pluginId)  
    }  
  }  
}
```

Properties

`pluginId`

- **Type:** string
- **Read-only:** Yes

- **Description:** Unique identifier for this plugin

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
Component.onCompleted: {  
  Logger.i("Plugin", "ID:", pluginApi.pluginId) // "my-plugin"  
}
```

pluginDir

- **Type:** string
- **Read-only:** Yes
- **Description:** Absolute path to plugin directory

```
readonly property string dir: pluginApi?.pluginDir || ""  
  
// Example: "/home/user/.config/noctalia/plugins/my-plugin"
```

pluginSettings

- **Type:** object
- **Read-only:** No
- **Description:** User settings object for this plugin

```
// Read setting  
readonly property string message: pluginApi?.pluginSettings?.message || ""  
  
// Write setting  
function updateMessage(newMessage) {  
  pluginApi.pluginSettings.message = newMessage  
  pluginApi.saveSettings() // Don't forget to save!  
}
```

 Caution

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

manifest

- **Type:** object
- **Read-only:** Yes
- **Description:** Plugin manifest data

```
// Access manifest data
readonly property string pluginName: pluginApi?.manifest?.name || ""
readonly property string pluginVersion: pluginApi?.manifest?.version || ""
readonly property var defaultSettings: pluginApi?.manifest?.metadata?.defaultSettings || {}

// Get default value if setting not set
readonly property string message:
  pluginApi?.pluginSettings?.message ||
  pluginApi?.manifest?.metadata?.defaultSettings?.message ||
  "Default"
```

currentLanguage

- **Type:** string
- **Read-only:** Yes
- **Description:** Current UI language code (e.g., "en", "es")

```
readonly property string lang: pluginApi?.currentLanguage || "en"

// Automatically updates when user changes language
```

pluginTranslations

- **Type:** object

- **Read-only:** Yes
- **Description:** Loaded translations for current language

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Don't access directly - use tr() function instead
// pluginApi.pluginTranslations contains the loaded translation data
```

mainInstance

- **Type:** Item (or null)
- **Read-only:** Yes
- **Description:** Reference to the instantiated Main.qml component

```
// Access the main instance from bar widget or panel
if (pluginApi?.mainInstance) {
    // Call a function defined in Main.qml
    pluginApi.mainInstance.doSomething()
}
```

Note

This property is null if your plugin doesn't have a main entry point in the manifest, or if Main.qml hasn't been loaded yet.

barWidget

- **Type:** Component (or null)
- **Read-only:** Yes
- **Description:** Reference to the bar widget Component

```
// Check if plugin has a bar widget
readonly property bool hasBarWidget: pluginApi?.barWidget !== null
```

Note

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

desktopWidget

- **Type:** Component (or null)
- **Read-only:** Yes
- **Description:** Reference to the desktop widget Component

```
// Check if plugin has a desktop widget  
readonly property bool hasDesktopWidget: pluginApi?.desktopWidget !== null
```

Note

This property is null if your plugin doesn't have a desktopWidget entry point in the manifest.

controlCenterWidget

- **Type:** Component (or null)
- **Read-only:** Yes
- **Description:** Reference to the control center widget Component

```
// Check if plugin has a control center widget  
readonly property bool hasCCWidget: pluginApi?.controlCenterWidget !== null
```

Note

This property is null if your plugin doesn't have a controlCenterWidget entry point in the manifest.

launcherProvider

- **Type:** Component (or null)
- **Read-only:** Yes

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Check if plugin has a launcher provider  
readonly property bool hasProvider: pluginApi?.launcherProvider !== null
```

Note

This property is null if your plugin doesn't have a launcherProvider entry point in the manifest.

panelOpenScreen

- **Type:** ShellScreen (or null)
- **Read-only:** Yes
- **Availability:** Only in Panel.qml
- **Description:** The screen that the panel was opened on

```
// In Panel.qml - get the screen this panel is displayed on  
var screen = pluginApi?.panelOpenScreen;  
if (screen && pluginApi?.manifest) {  
    BarService.openPluginSettings(screen, pluginApi.manifest);  
}
```

Note

This property is only available in Panel components. Use it when you need to reference the screen the panel is currently displayed on.

Functions

saveSettings()

Save plugin settings to disk.

Signature:

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
function saveSettings(): void
```

Usage:

```
function updateSetting(key, value) {  
  pluginApi.pluginSettings[key] = value  
  pluginApi.saveSettings()  
}
```

Example:

```
NButton {  
  text: "Save"  
  onClicked: {  
    pluginApi.pluginSettings.message = messageInput.text  
    pluginApi.pluginSettings.count = counterValue  
    pluginApi.saveSettings()  
  
    ToastService.showNotice("Settings saved!")  
  }  
}
```

Tip

Settings are saved to `~/.config/noctalia/plugins/<plugin-id>/settings.json`

openPanel(screen, buttonItem)

Open this plugin's panel on the specified screen.

Signature:

```
function openPanel(screen: ShellScreen, buttonItem?: Item): bool
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

- screen - The ShellScreen to open the panel on
- buttonItem - (Optional) The button/widget that triggered the panel. If provided, the panel will position itself near this button.

Returns: true if panel opened successfully, false otherwise

Usage:

```
// In BarWidget.qml - panel opens near the button
MouseArea {
    anchors.fill: parent
    onClicked: {
        pluginApi.openPanel(root.screen, root)
    }
}
```

Example:

```
NIconButton {
    id: myButton
    icon: "external-link"
    tooltip: "Open details"
    onClicked: {
        // Pass 'this' to position panel near the button
        if (pluginApi.openPanel(root.screen, this)) {
            Logger.i("Plugin", "Panel opened")
        } else {
            Logger.w("Plugin", "Failed to open panel")
        }
    }
}
```


**Tip**

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Note

This only works if your plugin provides a panel entry point in the manifest.

closePanel(screen)

Close this plugin's panel.

Signature:

```
function closePanel(screen: ShellScreen): bool
```

Parameters:

- screen - The ShellScreen to close the panel on

Returns: true if panel closed successfully, false otherwise

Usage:

```
// In Panel.qml - use pluginApi.panelOpenScreen
NButton {
    text: "Close"
    onClicked: {
        pluginApi.closePanel(pluginApi.panelOpenScreen)
    }
}
```

 Caution

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

~~pluginApi.panel.openScreen instead of root.screen. The screen property is not injected into plugin Panel components, so root.screen will be undefined.~~

togglePanel(screen, buttonItem)

Toggle this plugin's panel open/closed state.

Signature:

```
function togglePanel(screen: ShellScreen, buttonItem?: Item): bool
```

Parameters:

- screen - The ShellScreen to toggle the panel on
- buttonItem - (Optional) The button/widget that triggered the panel. If provided, the panel will position itself near this button.

Returns: true if operation succeeded, false otherwise

Usage:

```
// In BarWidget.qml or ControlCenterWidget.qml
NIconButtonHot {
    icon: "my-icon"
    onClicked: {
        pluginApi.togglePanel(root.screen, this)
    }
}
```



You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

the plugin's panel with a single click. Use this as the second parameter to have the panel open near the triggering button.

openLauncher(screen)

Open the launcher with this plugin's command prefix pre-filled. Only available for plugins that provide a launcherProvider entry point.

Signature:

```
function openLauncher(screen: ShellScreen): void
```

Parameters:

- screen - The ShellScreen to open the launcher on

Usage:

```
// In Main.qml IPC handler
pluginApi.withCurrentScreen(screen ⇒ {
  pluginApi.openLauncher(screen);
});
```

Note

The command prefix is determined from the commandPrefix field in your manifest's metadata, falling back to the plugin ID if not specified. For example, a plugin with "commandPrefix": "kaomoji" will open the launcher with >kaomoji pre-filled.

closeLauncher(screen)

Close the launcher.

Signature:

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Parameters:

- screen - The ShellScreen to close the launcher on

Usage:

```
pluginApi.withCurrentScreen(screen ⇒ {  
  pluginApi.closeLauncher(screen);  
});
```

toggleLauncher(screen)

Toggle the launcher open/closed with this plugin's command prefix. If the launcher is already open with a different command, it switches to this plugin's prefix instead of closing.

Signature:

```
function toggleLauncher(screen: ShellScreen): void
```

Parameters:

- screen - The ShellScreen to toggle the launcher on

Usage:

```
// In Main.qml IPC handler  
IpcHandler {
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
function toggle() {  
    pluginApi.withCurrentScreen(screen ⇒ {  
        pluginApi.toggleLauncher(screen);  
    });  
}  
}
```

Behavior:

- If the launcher is closed: opens it with the plugin's command prefix (e.g., >kaomoji)
- If the launcher is open with this plugin's prefix: closes the launcher
- If the launcher is open with a different prefix: switches to this plugin's prefix

Tip

toggleLauncher is the recommended way to handle IPC toggle commands for launcher provider plugins. It works correctly in both normal and overlay launcher modes.

tr(key, interpolations)

Translate a text key using plugin translations.

Signature:

```
function tr(key: string, interpolations: object = {}): string
```

Parameters:

- key - Translation key (supports dot notation for nested keys)
- interpolations - Object with placeholder values (optional)

Returns: Translated string, or ## key ## if translation not found

Usage:

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
NText {  
  text: pluginApi?.tr("panel.title") || "Title"  
}  
  
// With interpolations  
NText {  
  text: pluginApi?.tr("welcome.message", { name: userName }) || ""  
}
```

Translation file (i18n/en.json):

```
{  
  "panel": {  
    "title": "My Panel"  
  },  
  "welcome": {  
    "message": "Hello, {name}!"  
  }  
}
```

Example:

```
ColumnLayout {  
  NText {  
    text: pluginApi?.tr("settings.title") || "Settings"  
  }  
  
  NText {  
    text: pluginApi?.tr("status.count", { count: itemCount }) || ""  
    // With count=5: "You have 5 items"  
  }  
}
```

trp(key, count, defaultSingular, defaultPlural, interpolations)

Translate with plural forms.

Signature

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
function trp(  
  key: string,  
  count: number,  
  defaultSingular: string = "",  
  defaultPlural: string = "",  
  interpolations: object = {}  
): string
```

Parameters:

- key - Base translation key
- count - Number to determine singular/plural
- defaultSingular - Fallback singular text (optional)
- defaultPlural - Fallback plural text (optional)
- interpolations - Additional placeholder values (optional)

Returns: Translated string with correct plural form

Usage:

```
NText {  
  text: pluginApi?.trp(  
    "items.count",  
    itemCount,  
    "1 item",  
    "{count} items"  
  ) || ""  
}
```

Translation file (i18n/en.json):

```
{  
  "items": {
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
    "count_plural": "{count} items"  
  }  
}
```

Example:

```
property int fileCount: 5  
  
NText {  
  text: pluginApi?.trp(  
    "files.selected",  
    fileCount,  
    "1 file selected",  
    "{count} files selected"  
  ) || ""  
  // Output: "5 files selected"  
}
```

hasTranslation(key)

Check if a translation key exists.

Signature:

```
function hasTranslation(key: string): bool
```

Parameters:

- key - Translation key to check

Returns: true if translation exists, false otherwise

Usage:


```
if (pluginApi?.hasTranslation("optional.message")) {  
  text = pluginApi.tr("optional.message")
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
text = "Default message"  
}
```

Example:

```
function getLocalizedHelp() {  
  if (pluginApi?.hasTranslation("help.advanced")) {  
    return pluginApi.tr("help.advanced")  
  }  
  return pluginApi?.tr("help.basic") || "No help available"  
}
```

withCurrentScreen(callback)

Execute a callback with the current active screen. Useful in IPC handlers where you need a screen reference.

Signature:

```
function withCurrentScreen(callback: function): void
```

Parameters:

- callback - Function that receives the ShellScreen as its argument

Usage:

```
// In Main.qml IPC handler  
function toggle() {
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
    pluginApi.withCurrentScreen(screen ⇒ {  
        pluginApi.openPanel(screen);  
    });  
}  
}
```

Example:

```
lpcHandler {  
    target: "plugin:my-plugin"  
  
    function showPanel() {  
        pluginApi.withCurrentScreen(screen ⇒ {  
            pluginApi.openPanel(screen);  
        });  
    }  
  
    function closePanel() {  
        pluginApi.withCurrentScreen(screen ⇒ {  
            pluginApi.closePanel(screen);  
        });  
    }  
}
```

Tip

This is the preferred way to get a screen reference in IPC handlers, as it automatically selects the appropriate screen based on cursor position.

Accessing Noctalia Services

Plugins have full access to Noctalia services through QML imports.

Common Services

Settings

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Commons

// Access global Noctalia settings
readonly property bool isDarkMode: Settings.data.ui.darkMode
readonly property string barPosition: Settings.data.bar.position
readonly property string fontMain: Settings.data.ui.fontMain
```

Internationalization (l18n)

```
import qs.Commons

// Access global translations (not plugin translations)
NText {
    text: l18n.tr("common.save")
}

// Current language
readonly property string lang: l18n.langCode // "en", "es", etc.
```

Logger

```
import qs.Commons

Component.onCompleted: {
    Logger.i("PluginName", "Info message")
    Logger.d("PluginName", "Debug:", someValue)
    Logger.w("PluginName", "Warning!")
    Logger.e("PluginName", "Error occurred:", error)
}
```

Style

```
import qs.Commons
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
color: Style.capsuleColor
radius: Style.radiusM
height: Style.barHeight

Layout.margins: Style.marginL
}

NText {
  pointSize: Style.fontSizeM
}
```

Color

```
import qs.Commons

NText {
  color: Color.mOnSurface    // Primary text
  color: Color.mOnSurfaceVariant // Secondary text
  color: Color.mPrimary      // Accent
}

Rectangle {
  color: Color.mSurface
  color: Color.mSurfaceVariant
  color: "transparent"
}
```

Keybinds

```
import qs.Commons
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
Keys.onPressed: function(event) {
    if (Keybinds.checkKey(event, "escape", Settings)) {
        close();
        event.accepted = true;
    } else if (Keybinds.checkKey(event, "enter", Settings)) {
        submit();
        event.accepted = true;
    }
}

// Convert a key event to a keybind string (e.g. "Ctrl+A", "Shift+F5")
var keyStr = Keybinds.getKeybindString(event);

// Check for keybind conflicts (returns conflicting action name or null)
var conflict = Keybinds.getKeybindConflict(keyStr, "general.keybinds.keyEscape", Settings.data);
```

Common action names for `checkKey` include "up", "down", "left", "right", "enter", and "escape". More keybinds may be available - check [Settings.qml](#) under `general.keybinds` for the full list.

Tip

Use `Keybinds` instead of hardcoding `Qt.Key_*` checks. This ensures your plugin respects the user's custom keybind configuration.

UI Services

ToastService

```
import qs.Services.UI

ToastService.showNotice("Operation completed!")
ToastService.showError("Something went wrong")
```

TooltipService

```
import qs.Services.UI
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
hoverEnabled: true
onEntered: TooltipService.show(parent, "Tooltip text")
onExited: TooltipService.hide()
}
```

PanelService

```
import qs.Services.UI

// Open other panels
PanelService.openPanel("settings", root.screen)
PanelService.closePanel("launcher", root.screen)

// Check panel state
readonly property bool launcherOpen: PanelService.isPanelOpen("launcher", root.screen)

// Context menus (use these for proper cross-compositor support)
PanelService.showContextMenu(contextMenu, anchorItem, screen)
PanelService.closeContextMenu(screen)
```

Context Menu Functions

showContextMenu(contextMenu, anchorItem, screen)

Show a context menu anchored to a widget.

```
onRightClicked: {
  PanelService.showContextMenu(contextMenu, root, screen);
}
```

closeContextMenu(screen)

Close any open context menu on the specified screen.

```
onTriggered: action ⇒ {  
  contextMenu.close();
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Handle action...  
}
```

Tip

Always use `PanelService.showContextMenu()` instead of directly calling `contextMenu.open()`. This ensures proper input handling across all Wayland compositors.

System Services

AudioService

```
import qs.Services.System  
  
// Volume control  
readonly property real volume: AudioService.volume  
function setVolume(value) {  
  AudioService.setVolume(value)  
}  
  
// Mute  
readonly property bool isMuted: AudioService.muted  
function toggleMute() {  
  AudioService.setMuted(!AudioService.muted)  
}
```

BatteryService

```
import qs.Services.System  
  
readonly property real batteryPercent: BatteryService.percentage  
readonly property bool isCharging: BatteryService.charging  
readonly property string batteryIcon: BatteryService.icon
```

NetworkService

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

readonly property bool isConnected: NetworkService.connected
readonly property string networkName: NetworkService.ssid
readonly property int signalStrength: NetworkService.signalStrength

Complete Example

Here's a complete example using most of the Plugin API:


```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Commons
import qs.Widgets
import qs.Services.UI

Rectangle {
    id: root

    property var pluginApi: null
    property ShellScreen screen
    property string widgetId: ""
    property string section: ""
    property int sectionWidgetIndex: -1
    property int sectionWidgetsCount: 0

    // Get settings with defaults
    readonly property string displayText:
        pluginApi?.pluginSettings?.text ||
        pluginApi?.manifest?.metadata?.defaultSettings?.text ||
        "Default"

    readonly property int counter:
        pluginApi?.pluginSettings?.counter || 0

    implicitWidth: row.implicitWidth + Style.marginM * 2
    implicitHeight: Style.barHeight

    color: Style.capsuleColor
    radius: Style.radiusM

    RowLayout {
        id: row
        anchors.centerIn: parent
        spacing: Style.marginS

        NIcon {
            icon: "plugin"
```

```
    color: Color.mPrimary
  }
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
    text: pluginApi?.tr("widget.title") || "Plugin"
    color: Color.mOnSurface
    pointSize: Style.fontSizeS
  }
```

```
NText {
    text: root.counter.toString()
    color: Color.mPrimary
    pointSize: Style.fontSizeS
    font.weight: Font.Bold
    visible: root.counter > 0
  }
}
```

```
MouseArea {
    anchors.fill: parent
    hoverEnabled: true
    acceptedButtons: Qt.LeftButton | Qt.RightButton
```

```
    cursorShape: Qt.PointingHandCursor
```

```
    onEntered: {
        root.color = Qt.lighter(Style.capsuleColor, 1.1)
```

```
        // Show tooltip
        var tooltip = pluginApi?.trp(
            "widget.clicks",
            root.counter,
            "Click to increment",
            "{count} clicks"
        ) || ""
        TooltipService.show(root, tooltip)
    }
```

```
    onExited: {
        root.color = Style.capsuleColor
```

```
TooltipService.hide()
}
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
if (mouse.button === Qt.LeftButton) {
    // Increment counter
    var newCount = root.counter + 1
    pluginApi.pluginSettings.counter = newCount
    pluginApi.saveSettings()

    Logger.i(pluginApi.pluginId, "Counter incremented to:", newCount)
    ToastService.showNotice(
        pluginApi?.tr("widget.incremented") || "Incremented!"
    )
} else if (mouse.button === Qt.RightButton) {
    // Open panel near this widget
    if (pluginApi.openPanel(root.screen, root)) {
        Logger.i(pluginApi.pluginId, "Panel opened")
    }
}
}

Component.onCompleted: {
    Logger.i("Plugin", "Widget loaded")
    Logger.d("Plugin", "ID:", pluginApi?.pluginId)
    Logger.d("Plugin", "Version:", pluginApi?.manifest?.version)
    Logger.d("Plugin", "Language:", pluginApi?.currentLanguage)
    Logger.d("Plugin", "Counter:", root.counter)
}
}
```

See Also

- [Getting Started](#) - Create your first plugin
- [Bar Widget](#) - Bar widget development
- [Desktop Widget](#) - Desktop widget development

- [Control Center Widget](#) - Control center widget development
- [Launcher Provider](#) - Launcher provider development

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

- [Settings UI](#) - Settings UI development
- [Translations](#) - i18n guide
- [Manifest Reference](#) - Plugin configuration
- [Plugin Overview](#) - Plugin system overview